

02-脚手架+指令

一、脚手架

1. Vue代码开发方式

1.1 思考

相比直接 script 引入 vue 源码, 有没有更好的方式编写vue代码呢?

1.2 两种开发方式对比

1.2.1 传统开发模式

基于html文件开发Vue, 类似jQuery的使用, `<script src="vue.js"></script>`

✓ VUE-TRADITION-WAY

```
# index.css
<> index.html
JS index.js
```

- 优点: 简单、上手快
- 缺点: 功能单一、开发体验差

1.2.2 工程化开发模式

在 构建工具(Vite/Webpack)环境下开发Vue, 这是最推荐的、也是企业采用的方式

✓ VUE-ENGINEERING-WAY

```
> node_modules
> public
✓ src
  > assets
  > components
  ▼ App.vue
  JS main.js
  # style.css
  .gitignore
  <> index.html
  {} package.json
  ⓘ README.md
  JS vite.config.js
  👤 yarn.lock
```

- 优点:

- a. 功能全面
- b. 开发体验好

...

- 缺点：
 - a. 目录结构复杂
 - b. 理解难度提升

1.3 总结

1. Vue代码有哪两种开发方式？

- 答：1. 传统开发方式，直接用script引入vue.js文件
2. 在工程化环境下，借助Vite对Vue代码进行打包 (推荐)

2. 工程化环境下开发Vue代码，相比传统方式，有哪些优势？

- 答：1. 功能全面
2. 开发体验好

2. 准备工程化环境

2.1 安装工具

2.1.1 安装Nodejs

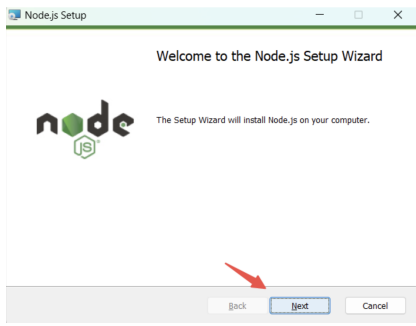
注意：安装18.3或更高版本的 Nodejs

官网：<https://nodejs.org/en/>

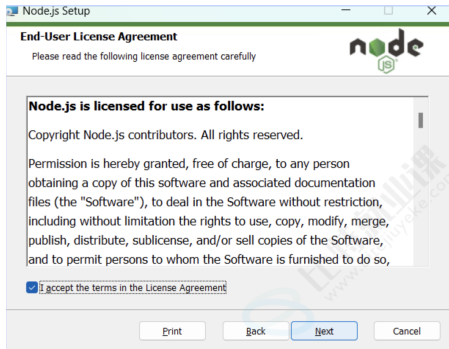
- 点击下载长期版



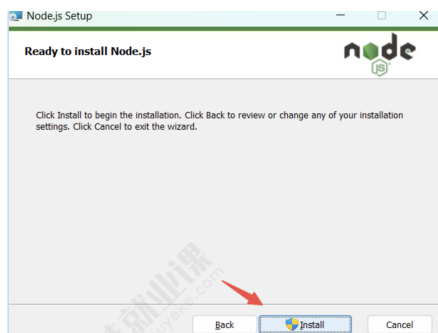
- 双击安装程序、并点击下一步



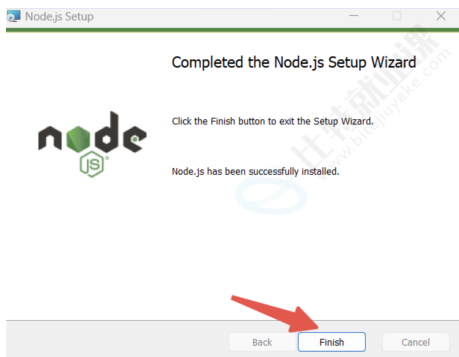
- 选择接受协议



- 点击下一步
- 点击install进行安装



- 安装完成点击finish



- 打开命令行窗口

```
1 node -v
2 npm -v
```

- npm换源

```
1 // 查看 npm 源
2 npm config get registry
3 // 默认是指向 https://registry.npmjs.org/, 也就是官方源
4
5 // 为了提高npm下载速度, 可以给npm换源
6 // 国内源有很多, 我这里用淘宝源吧。毕竟是大公司, 会比较稳定
7 npm config set registry https://registry.npmmirror.com
8
9 // 再一次查看 npm 源
10 npm config get registry
```

2.1.2 安装yarn或pnpm

yarn和pnpm、还有npm三者的功能类似, 都是包管理工具,
用来下载或删除模块包, 性能上yarn和pnpm优于npm

```
1 # windows系统
2 npm install yarn -g
3
4 npm install pnpm -g
5
6 -----
7
8 # mac系统
9 sudo npm install yarn -g
10
11 sudo npm install pnpm -g
```

- 检测是否安装成功

```
1 yarn -v
2
3 pnpm -v
```

2.2 总结

1. 准备工程化环境要安装哪些工具?

答：node / yarn | pnpm

3. 创建Vue工程化项目

3.1 脚手架

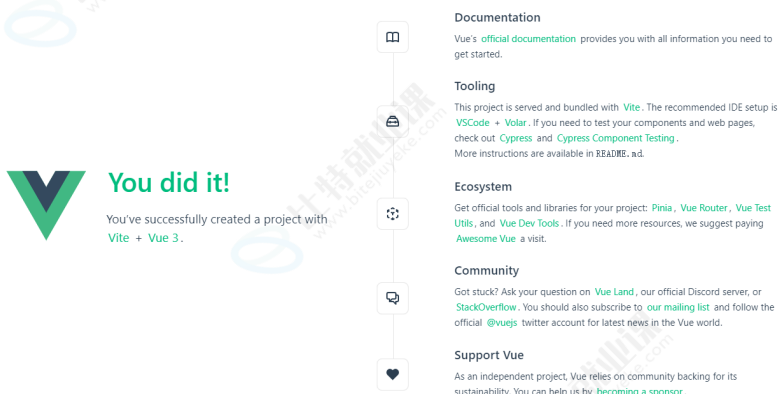
一个保证各项工作顺利开展的平台。好处就是拿来就用，零配置，基于它我们来学习Vue



3.2 创建步骤

1. 选定一个存放位置，比如选择桌面，根据自己情况，选择D盘或E盘等
2. 执行命令 `npm create vue@latest`，会安装并执行 `create-vue`，它是Vue 官方的项目脚手架工具
3. 进入项目根目录：`cd 项目名称`
4. 安装vue等模块依赖：`npm i`
5. 启动项目：`npm run dev`，会开启一个本地服务器
6. 浏览器网址栏输入：<http://localhost:5173>

最后呈现的界面效果如下，说明项目创建并启动成功了



3.3 总结

1. Vue脚手架是什么？有什么好处？

答：一个写Vue代码的环境(基础), 拿来就用, 零配置

2. 如何创建一个脚手架项目？

答：npm create vue@latest

3. 如何启动项目？

答：1、npm dev , 启动一个本地服务器

2、浏览器输入 <http://localhost:5173>

4. 认识脚手架目录及文件

4.1 目录和文件解读



4.2 总结

1. 脚手架项目中，几个主要的文件及作用？

答：1、node_modules —— 第三方模块包

2、package.json —— 项目管理文件

3、src/main.js -- 整个项目打包的入口

4、src/App.vue -- Vue代码的入口(根组件)

5、index.html -- 项目入口网页

2. 我们今后Vue代码写哪个目录下？

答：src 目录, src下的所有代码会被 vite 打包 成 css/js/img,

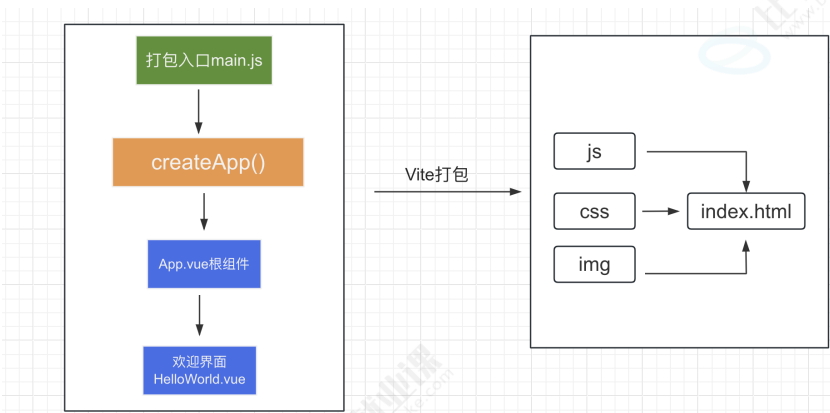
然后 交给index.html , 最终通过浏览器呈现在用户眼前

5. 分析3个入口文件的关系

5.1 目标

要知道3个入口文件的关系，以及我们编写的源代码是如何通过浏览器展示出来的

5.2 图解



5.3 总结

1. main.js、App.vue、index.html 三个文件的作用？

答：1、main.js — 项目打包的入口 — 创建应用

2、App.vue — Vue代码的入口(根组件)

3、index.html — 项目的入口网页

2. main.js、App.vue、index.html 三者的关系是什么？

答：1、App.vue(Vue入口) => main.js(项目打包入口) => index.html(浏览器入口)

2、main.js 是 Vue 代码通向网页代码的桥梁，非常关键

6. Vue单文件

6.1 思考

代码写一起、担心class类名、js变量名 重名冲突？Vue中如何避免呢？

6.2 vue单文件介绍

1. vue推荐采用 .vue 的文件来开发项目

2. 一个vue文件通常有3部分组成，script(JS) + template(HTML) + style(CSS)

3. 一个vue文件是 独立的模块，作用域互不影响

4. style配合 scoped 属性，保证样式只针对当前 template 内的标签生效

5. vue文件会被 vite打包成js、css 等，最终交给index.html 通过浏览器呈现效果

6.3 总结

1. vue文件的组成是什么？

答：三部分组成 = script(JS)+template(HTML)+style(CSS)

2. vue单文件的好处？

答：独立的作用域, 不用担心 JS 变量名、CSS 选择器名称冲突

7. 清理目录结构

7.1 目标

为了便于后续学习Vue代码，这里需要清楚默认不需要的目录结构

7.2 步骤

1. 删除 assets 文件夹
2. 删除 components 文件夹
3. 清除 App.vue 的内容
4. 清除 main.js 的内容

7.3 补充内容

在 **App.vue** 中

```
1  <script setup></script>
2
3  <template>
4    App根组件
5  </template>
6
7  <style></style>
```

在 **main.js** 中

```
1  import { createApp } from 'vue'
2  import App from './App.vue'
3
4  createApp(App).mount('#app')
```

8. setup简写+插值+响应式

8.1 完整写法


```
1  <script>
2    export default {
3      setup() {
4        // ...
5        const msg = 'Hello Vue3+Vite'
6
7        return {
8          msg
9        }
10     }
11   }
12 </script>
13
14 <template>
15   <h1>{{ msg }}</h1>
16 </template>
17
18 <style></style>
```

8.2 简化写法(推荐)

```
1  <script setup>
2    const msg = 'Hello Vue3+Vite'
3  </script>
4
5  <template>
6    <h1>{{ msg }}</h1>
7  </template>
8
9  <style></style>
```

8.3 练习代码示例

```
1  <script setup>
2    // 导入响应式函数
3    import { reactive, ref } from 'vue'
4
5    // 字符串
6    const msg = ref('Hello Vue3+Vite')
7
8    // 对象
9    const obj = reactive({
10      name: '小vue',
```

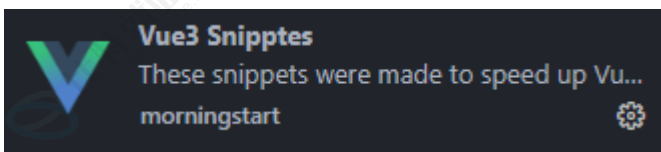
```

11     age: 9
12   })
13
14   // 函数
15   function fn() {
16     return 100
17   }
18 </script>
19
20 <template>
21   <!-- 1. 直接放变量 -->
22   <h1>{{ msg }}</h1>
23   <!-- 2. 对象.属性 -->
24   <p>我叫 {{ obj.name }}, 今年 {{ obj.age }} 岁</p>
25   <!-- 3. 三元表达式 -->
26   <p>{{ obj.age >= 18 ? '已成年' : '未成年' }}</p>
27   <!-- 3. 算数运算 -->
28   <p>来年我就 {{ obj.age + 1 }} 岁了</p>
29   <!-- 4. 函数的调用 -->
30   <p>fn的返回值是: {{ fn() }}</p>
31   <!-- 4. 方法的调用 -->
32   <p>{{ msg }} 中含有 {{ msg.split(' ').length }} 个单词</p>
33 </template>

```

8.4 安装VSCode插件

为了今后可以快速生成 vue 文件的三部分组成, 建议安装VSCode的插件 **Vue3 Snippets**, 然后在 vue 文件中输入 **vbase** 即可快速生成模版



8.5 总结

1. 推荐使用 **setup** 简化写法

```

1 <script setup></script>
2
3 <template></template>
4
5 <style></style>

```

二、指令

1. Vue中的常用指令

1.1 概念

指令（Directives）是 Vue 提供的带有 v- 前缀的特殊标签属性, 用来增强标签的能力

1.2 作用

提高标签数据渲染的能力

1.3 分类

vue3 中的指令按照不同的用途可以分为如下 6 大类：

- 内容渲染指令（v-html、v-text）
- 属性绑定指令（v-bind）
- 事件绑定指令（v-on）
- 条件渲染指令（v-show、v-if、v-else、v-else-if）
- 列表渲染指令（v-for）
- 双向绑定指令（v-model）

指令是 vue 开发中最基础、最常用、最简单的知识点。

2. 内容渲染指令

2.1 目标

知道两个指令的作用和区别

2.2 作用

内容渲染指令用来辅助开发者渲染 DOM 元素的文本内容。

常用的内容渲染指令有如下2 个：

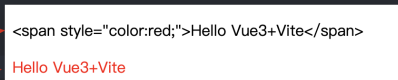
- v-text（类似innerText）
 - 使用语法： `<p v-text="表达式"></p>`，意思是将 表达式的 值渲染到 p 标签中
 - 类似 innerText，使用该语法，会覆盖 p 标签原有内容
- v-html（类似 innerHTML）
 - 使用语法： `<p v-html="表达式"></p>`，意思是将 表达式的 值渲染到 p 标签中
 - 类似 innerHTML，使用该语法，会覆盖 p 标签原有内容
 - 类似 innerHTML，使用该语法，能够将HTML标签的样式呈现出来。

```

<script setup>
  import { ref } from 'vue' 556.4k (gzipped: 178.4k)
  const str = ref('<span style="color:red;">Hello Vue3+Vite</span>')
</script>

<template>
  <div>
    <p v-text="str"></p>
    <p v-html="str"></p>
  </div>
</template>

```



2.3 代码示例

```

1  <script setup>
2    import { ref } from 'vue'
3    const str = ref('<span style="color:red;">Hello Vue3+Vite</span>')
4  </script>
5
6  <template>
7    <div>
8      <p v-text="str"></p>
9      <p v-html="str"></p>
10   </div>
11 </template>
12
13 <style scoped></style>

```

2.4 总结

1. v-text和v-html有什么作用？

答：把表达式的值展示在双标签中

2. 二者有何区别？

答：v-text不解析标签、v-html解析标签

3. 属性绑定指令

3.1 目标

掌握vue属性绑定的语法

3.2 作用

把 表达式的结果 与标签的 属性动态绑定

3.3 语法

- v-bind:属性名="表达式"
- 可简写成: 属性名="表达式"

```
<!-- v-bind: 标签属性的动态绑定 -->
<a v-bind:href="url">百度一下</a>
<!-- 简写 -->
<a :href="url">百度一下</a>
```

比如，有一个图片，它的 `src` 属性值，是一个图片地址，这个地址是ref定义的，则可以这样设置属性值：

- ``
- `` (v-bind可以省略)

3.4 代码示例

```
1 <script setup>
2   import { ref } from 'vue'
3   const url = ref('http://www.baidu.com')
4 </script>
5
6 <template>
7   <div>
8     <!-- v-bind: 标签属性动态绑定 -->
9     <a v-bind:href="url">百度一下</a>
10    <!-- 简写 -->
11    <a :href="url">百度一下</a>
12  </div>
13 </template>
```

3.5 总结

1. 如何给DOM标签的属性，绑定Vue表达式？

答：1、借助 `v-bind` 指令的简写，语法 `:属性名="表达式"`

2、注意 属性名前的 冒号不能丢

4. 事件绑定指令

4.1 目标

掌握vue事件绑定语法

4.2 作用

给元素绑定事件

4.3 语法(3种写法)

以给 `button` 添加点击事件为例

- 1 `<button v-on:事件名="内联语句">按钮</button>`
- 2 `<button v-on:事件名="处理函数">按钮</button>`
- 3 `<button v-on:事件名="处理函数(实参列表)">按钮</button>`

```
<script setup>
import { ref } from 'vue'

const count = ref(0)

const increase = () => {
  count.value++
}

const add = (n) => {
  count.value += n
}
</script>

<div>
<p>{{ count }}</p>
<!-- 1. 内联语句 -->
<button v-on:click="count++">+1</button>
<!-- 2. 调用无参函数 -->
<button v-on:click="increase">+1</button>
<!-- 3. 调用有参函数 -->
<button v-on:click="add(3)">+3</button>
<!-- 4. 简写 @click -->
<button @click="add(5)">+5</button>
</div>
```

- `v-on:` 可以简写为 `@`
- 函数需要在 `script` 下声明

4.4 代码示例

```
1 <script setup>
2   import { ref } from 'vue'
3
4   // 响应式数据 计数器
5   const count = ref(0)
6
7   // 无参函数
8   const increase = () => {
9     count.value++
10  }
11
12  // 有参函数
13  const add = (n) => {
14    count.value += n
15  }
16 </script>
17
18 <template>
19   <div>
20     <p>{{ count }}</p>
21     <!-- 1. 内联语句 -->
22     <button v-on:click="count++">+1</button>
23     <!-- 2. 调用无参函数 -->
24     <button v-on:click="increase">+1</button>
25     <!-- 3. 调用有参函数 -->
26     <button v-on:click="add(3)">+3</button>
```

```
27     <button v-on:click="add(5)">+5</button>
28     <!-- 4. 简写 @click -->
29     <button @click="add(5)">+5</button>
30 </div>
31 </template>
```

4.5 总结

1. 如何给DOM标签绑定事件？

答：借助 `v-on` 指令的简写，语法 `@事件名="函数"`

2. 如何给事件函数传参数？

答： `@事件名="函数(实参列表)"`

3. 函数在哪声明？

答：script 标签下直接声明

5. 条件渲染指令

5.1 目标

掌握条件判断指令，用来辅助开发者控制 DOM 的显示与隐藏。

5.2 区别

1. v-show

- 作用：控制元素显示隐藏
- 语法： `v-show = "布尔表达式"` 表达式值为 true 显示，false 隐藏
- 原理：切换 `display:none` 控制显示隐藏
- 场景：频繁切换显示隐藏的场景

2. v-if

- 作用：控制元素显示隐藏（条件渲染）
- 语法： `v-if = "布尔表达式"` 表达式值 true 显示，false 隐藏
- 原理：基于条件判断，创建 或 移除 元素
- 场景：要么显示，要么隐藏，不频繁切换的场景

3. v-else 和 v-else-if

- 作用：辅助v-if进行判断渲染
- 语法： `v-else v-else-if="表达式"`
- 需要紧接着v-if使用

```

<script setup>
import { ref } from 'vue'

// 控制盒子是否可见
const visible = ref(true)

// 模拟是否登录
const islogin = ref(true)

// 成绩
const mark = ref(55)
</script>

```

```

<template>
<div>
<!-- v-show -->
<div class="red" v-show="visible"></div>
<!-- v-if -->
<div class="green" v-if="visible"></div>

<hr />

<!-- v-if配合v-else做双分支渲染 -->
<div v-if="islogin">xxx, 欢迎回来</div>
<div v-else>你好, 请登录</div>

<hr />

<!-- v-if配合v-else-if和v-else做多分支渲染 -->
<div v-if="mark >= 90">优秀</div>
<div v-else-if="mark >= 70">良好</div>
<div v-else>差</div>
</div>
</template>

```

5.3 代码示例

```

1  <script setup>
2    import { ref } from 'vue'
3    // 是否可见
4    const visible = ref(true)
5    // 是否登录
6    const isLogin = ref(true)
7    // 成绩
8    const mark = ref(100)
9  </script>
10
11  <template>
12    <!-- v-show -->
13    <div class="red" v-show="visible"></div>
14    <!-- v-if -->
15    <div class="green" v-if="visible"></div>
16
17    <hr>
18
19    <!-- 双分支的条件渲染 -->
20    <div v-if="isLogin">xxx, 欢迎回来</div>
21    <div v-else>你好, 请登录</div>
22
23    <hr>
24
25    <!--
26      多分支的条件渲染:
27      1. 90及其以上优秀
28      2. 70到90之间良好
29      3. 其他的差
30    -->
31    <div v-if="mark >= 90">优秀</div>
32    <div v-else-if="mark >= 70">良好</div>
33    <div v-else>差</div>
34  </template>
35
36  <style scoped>

```



```
37     .red, .green {
38         width: 200px;
39         height: 200px;
40     }
41     .red {
42         background: red;
43     }
44
45     .green {
46         background: green;
47     }
48 </style>
```

5.4 总结

1. vue中如何控制标签显示/隐藏?

答：借助 `v-show` 和 `v-if` 指令关联布尔表达式，true显示; false隐藏

2. 二者区别是什么?

答：1、v-show是通过控制css的display属性

2、v-if直接从DOM树上移除或创建; 并且v-if还可以配合v-else-if/v-else进行多分支条件渲染

6. 案例轮播图

6.1 效果



6.2 静态代码(可复制)

```
1 <script setup>
2   const images = [
3     'https://bit-1256306791.cos.ap-
4     chengdu.myqcloud.com/fffa0f9be4b071da45d9a0c2.jpg',
5     'https://bit-1256306791.cos.ap-
6     chengdu.myqcloud.com/fffa0fa3e4b071da45d9a0c3.jpg',
```

```
5     'https://bit-1256306791.cos.ap-
chengdu.myqcloud.com/fffa10ace4b071da45d9a0c4.jpg'
6 ]
7 </script>
8
9 <template>
10     <div class="banner">
11         
13         <div class="ctrl">
14             <a
15                 href="javascript:;"
16                 class="btn prev"
17                 @click="prev"
18                 >&lt;</a>
19             ><a
20                 href="javascript:;"
21                 class="btn next"
22                 @click="next"
23                 >&gt;</a>
24         >
25     </div>
26 </div>
27 </template>
28
29 <style>
30     a {
31         text-decoration: none;
32         color: #000;
33     }
34
35     .banner {
36         position: relative;
37         width: 1200px;
38         height: 337px;
39         margin: 150px auto;
40     }
41     .btn {
42         position: absolute;
43         top: 50%;
44         width: 30px;
45         height: 30px;
46         line-height: 30px;
47         margin-top: -15px;
48         font-size: 14px;
49         text-align: center;
```

```
50     background: rgba(0, 0, 0, 0.3);
51     color: #fff;
52     border-radius: 50%;
53   }
54   .btn.prev {
55     left: 15px;
56   }
57   .btn.next {
58     right: 15px;
59   }
60 </style>
```

6.3 完整代码

```
1  <script setup>
2    import { ref } from 'vue'
3    const i = ref(0)
4    const images = [
5      'https://bit-1256306791.cos.ap-
chengdu.myqcloud.com/fffa0f9be4b071da45d9a0c2.jpg',
6      'https://bit-1256306791.cos.ap-
chengdu.myqcloud.com/fffa0fa3e4b071da45d9a0c3.jpg',
7      'https://bit-1256306791.cos.ap-
chengdu.myqcloud.com/fffa10ace4b071da45d9a0c4.jpg'
8    ]
9
10   const prev = () => {
11     i.value--
12     if (i.value <= -1) {
13       i.value = images.length - 1
14     }
15   }
16
17   const next = () => {
18     console.log('next')
19     i.value++
20     if (i.value >= images.length) {
21       i.value = 0
22     }
23   }
24 </script>
25
26 <template>
27   <div class="banner">
28     
```

```
29     <div class="ctrl">
30         <a
31             href="javascript:;"
32             class="btn prev"
33             @click="prev"
34             >&lt;</a>
35         ><a
36             href="javascript:;"
37             class="btn next"
38             @click="next"
39             >&gt;</a>
40     >
41 </div>
42 </div>
43 </template>
44
45 <style>
46     a {
47         text-decoration: none;
48         color: #000;
49     }
50
51     .banner {
52         position: relative;
53         width: 1200px;
54         height: 337px;
55         margin: 150px auto;
56     }
57     .btn {
58         position: absolute;
59         top: 50%;
60         width: 30px;
61         height: 30px;
62         line-height: 30px;
63         margin-top: -15px;
64         font-size: 14px;
65         text-align: center;
66         background: rgba(0, 0, 0, 0.3);
67         color: #fff;
68         border-radius: 50%;
69     }
70     .btn.prev {
71         left: 15px;
72     }
73     .btn.next {
74         right: 15px;
75     }
```

7. 案例可折叠面板

7.1 效果

可折叠面板

自由与爱情	收起
生命诚可贵， 爱情价更高。 若为自由故， 两者皆可抛。	

7.2 静态代码(可复制)

```
1  <script setup></script>
2
3  <template>
4    <h3>可折叠面板</h3>
5    <div class="panel">
6      <div class="title">
7        <h4>自由与爱情</h4>
8        <span class="btn"> 收起 </span>
9      </div>
10     <div class="container">
11       <p>生命诚可贵,</p>
12       <p>爱情价更高。</p>
13       <p>若为自由故,</p>
14       <p>两者皆可抛。</p>
15     </div>
16   </div>
17 </template>
18
19 <style lang="scss">
20   body {
21     background-color: #ccc;
22   }
```

```

23
24 #app {
25     width: 400px;
26     margin: 20px auto;
27     padding: 1em 2em 2em;
28     box-shadow: 3px 3px 3px rgba(0, 0, 0, 0.5);
29     background-color: #fff;
30 }
31 #app h3 {
32     text-align: center;
33 }
34
35 .panel {
36     .title {
37         display: flex;
38         justify-content: space-between;
39         align-items: center;
40         padding: 0 1em;
41         border: 1px solid #ccc;
42     }
43     .title h4 {
44         line-height: 2;
45         margin: 0;
46     }
47     .container {
48         border: 1px solid #ccc;
49         padding: 0 1em;
50     }
51     .btn {
52         cursor: pointer;
53     }
54 }
55 </style>

```

7.3 完整代码

```

1 <script setup>
2   import { ref } from 'vue'
3
4   // 控制盒子的显示或隐藏
5   const visible = ref(true)
6 </script>
7
8 <template>
9   <h3>可折叠面板</h3>

```

```

10   <div class="panel">
11     <div class="title">
12       <h4>自由与爱情</h4>
13       <span
14         class="btn"
15         @click="visible = !visible">
16         {{ visible ? '收起' : '展开' }}
17       </span>
18     </div>
19     <div
20       class="container"
21       v-show="visible">
22       <p>生命诚可贵,</p>
23       <p>爱情价更高.</p>
24       <p>若为自由故,</p>
25       <p>两者皆可抛.</p>
26     </div>
27   </div>
28 </template>
29
30 <style lang="scss">
31   // 注意: 当前App编写样式用到了 sass 预处理器语法,
32   // 脚手架工程默认不支持 css 预处理器的编译, 需要安装相应的模块
33   // npm i -D sass-embedded
34   // npm i sass-embedded -D
35 </style>

```

8. 列表渲染指令

8.1 目标

基于数组进行 v-for 列表渲染

8.2 语法

v-for 指令需要使用 `(item, index) in 目标结构` 形式的特殊语法, 其中:

- item: 数组中的每一项
- index: 每一项的索引, 不需要可以省略
- 目标结构: 被遍历的 数组/对象/数字

```

1  <script setup>
2    import { ref } from 'vue'
3    // 数字数组
4    const nums = ref([11, 22, 33, 44])

```

```
5
6 // 商品列表
7 const goodsList = ref([
8   { id: 1, name: '篮球', price: 299 },
9   { id: 2, name: '足球', price: 99 },
10  { id: 3, name: '排球', price: 199 }
11 ])
12
13 // 准备对象
14 const obj = {
15   id: 10001,
16   name: 'bit',
17   age: 9
18 }
19 </script>
20 <template>
21   <div>
22     <ul>
23       <!-- 遍历数字数组 -->
24       <li v-for="(item, index) in nums">{{ item }} => {{ index }}</li>
25     </ul>
26     <div class="goods-list">
27       <!-- 遍历对象数组 -->
28       <div
29         class="goods-item"
30         v-for="item in goodsList">
31         <p>id = {{ item.id }}</p>
32         <p>name = {{ item.name }}</p>
33         <p>price = {{ item.price }}</p>
34       </div>
35       <ul>
36         <!-- 遍历对象 -->
37         <li v-for="(value, key, index) in obj">
38           {{ value }} => {{ key }} => {{ index }}
39         </li>
40       </ul>
41     </div>
42     <ul>
43       <!-- 遍历数字 -->
44       <li v-for="(item, index) in 5">{{ item }} => {{ index }}</li>
45     </ul>
46   </div>
47 </div>
48 </template>
49
50 <style scoped></style>
```


8.3 总结

1. v-for如何循环生成列表？

答：1、想循环谁就把 v-for 添加给谁
2、语法: v-for="(item, index) in 目标数据"

2. v-for可以遍历的哪些目标数据？

答：数组 / 对象 / 数字

9. 案例比特人的橱窗

9.1 效果

比特人的橱窗

商品名称	商品价格	操作
biangbiang面	13	删除
葫芦头	20	删除
冰峰	2	删除
肉夹馍	11	删除
羊肉泡馍	18	删除
暂无数据啦		

9.2 静态代码

```
1  <script setup>
2    // 商品列表
3    const goodsList = [
4      { id: 101, name: 'biangbiang面', price: 13 },
5      { id: 102, name: '葫芦头', price: 20 },
6      { id: 103, name: '冰峰', price: 2 },
7      { id: 104, name: '肉夹馍', price: 11 },
8      { id: 105, name: '羊肉泡馍', price: 18 }
9    ]
10 </script>
11
12 <template>
13   <table>
```

```
14     border="1"
15     cellspacing="0"
16     cellpadding="0">
17     <caption>
18         <h3>比特人的橱窗</h3>
19     </caption>
20     <thead>
21         <tr>
22             <th>商品名称</th>
23             <th>商品价格</th>
24             <th>操作</th>
25         </tr>
26     </thead>
27     <tbody>
28         <tr>
29             <td>油泼面</td>
30             <td>10</td>
31             <td>
32                 <button>删除</button>
33             </td>
34         </tr>
35     </tbody>
36     <tfoot>
37         <tr>
38             <td colspan="3">暂无数据啦</td>
39         </tr>
40     </tfoot>
41 </table>
42 </template>
43
44 <style lang="scss">
45     table {
46         width: 500px;
47         margin: 150px auto;
48         text-align: center;
49         font-family: 'Courier New', Courier, monospace;
50
51         tr {
52             height: 40px;
53         }
54
55         tfoot {
56             height: 100px;
57         }
58     }
59 </style>
```

9.3 完整代码

```
1  <script setup>
2    import { ref } from 'vue'
3    // 商品列表
4    const goodsList = ref([
5      { id: 101, name: 'biangbiang面', price: 13 },
6      { id: 102, name: '葫芦头', price: 20 },
7      { id: 103, name: '冰峰', price: 2 },
8      { id: 104, name: '肉夹馍', price: 11 },
9      { id: 105, name: '羊肉泡馍', price: 18 }
10   ])
11
12   // 数组的删除方法:
13   //  unshift: 头部删除
14   //  pop: 尾部删除
15   //  splice(startIndex, delCount): 任意位置删除或添加
16
17   // 删除
18   const del = (i) => {
19     if (window.confirm('确认删除么?')) {
20       goodsList.value.splice(i, 1)
21     }
22   }
23 </script>
24
25 <template>
26   <table
27     border="1"
28     cellpadding="0"
29     cellspacing="0">
30     <caption>
31       <h3>比特人的橱窗</h3>
32     </caption>
33     <thead>
34       <tr>
35         <th>商品名称</th>
36         <th>商品价格</th>
37         <th>操作</th>
38       </tr>
39     </thead>
40     <tbody>
41       <tr v-for="(item, index) in goodsList">
42         <td>{{ item.name }}</td>
43         <td>{{ item.price }}</td>
44         <td>
```

```

45         <button @click="del(index)">删除</button>
46     </td>
47 </tr>
48 </tbody>
49 <tfoot v-if="goodsList.length === 0">
50     <tr>
51         <td colspan="3">暂无数据啦</td>
52     </tr>
53 </tfoot>
54 </table>
55 </template>

```

10. v-for中的key

10.1 目标

知道为什么要给v-for添加key

10.2 语法

:key="唯一值"

10.3 作用

给列表项添加的唯一标识, 便于Vue进行列表项的正确排序复用, 因为Vue 的默认行为会尝试原地修改元素 (就地复用)

10.4 代码示例

```

1  <script setup>
2      import { ref } from 'vue'
3
4      // 图书列表
5      const bookList = ref([
6          { id: 1, name: '《红楼梦》', author: '曹雪芹' },
7          { id: 2, name: '《西游记》', author: '吴承恩' },
8          { id: 3, name: '《三国演义》', author: '罗贯中' },
9          { id: 4, name: '《水浒传》', author: '施耐庵' }
10     ])
11     // 删除
12     const onDelete = (i) => {
13         // i: 当前点击的下标
14
15         // 删除前先确认
16         if (window.confirm('确定删除么?')) {
17             // 调用 splice 进行删除

```

```

18     bookList.value.splice(i, 1)
19   }
20 }
21 </script>
22
23 <template>
24   <h3>比特人的书架</h3>
25   <ul>
26     <!-- 无key -->
27     <li v-for="(item, index) in bookList">
28       <span>{{ item.name }}</span>
29       <span>{{ item.author }}</span>
30       <button @click="onDel(index)">删除</button>
31     </li>
32   </ul>
33
34   <ul>
35     <!-- 有key且为id -->
36     <li
37       v-for="(item, index) in bookList"
38       :key="item.id">
39       <span>{{ item.name }}</span>
40       <span>{{ item.author }}</span>
41       <button @click="onDel(index)">删除</button>
42     </li>
43   </ul>
44 </template>
45
46 <style>
47   #app {
48     width: 400px;
49     margin: 100px auto;
50   }
51   ul {
52     list-style: none;
53   }
54   ul li {
55     display: flex;
56     justify-content: space-around;
57     padding: 10px 0;
58     border-bottom: 1px solid #ccc;
59   }
60 </style>

```

10.5 注意

1. key 的类型只能是 数字 或 字符串
2. key 的值必须 唯一, 不能重复
3. 推荐用 id 作为 key (因为id唯一), 不推荐用 index 作为 key(会变化)

10.6 总结

1. 为什么要给v-for所在的元素添加key?

答: 最大限度的复用DOM、从而提高DOM的更新性能

2. key的类型?

答: 数字 / 字符串

3. key的选择?

答: 首选每次循环的 id、其次是下标

11. 双向绑定指令

11.1 介绍

所谓双向绑定就是:

1. 数据变了 -> 视图的变化
2. 视图变了 -> 数据的变化

11.2 作用

作用在 **表单元素** (input、select、radio、checkbox) 上, 实现数据双向绑定, 从而可以快速 **获取** 或 **设置** 表单元素的值

11.3 语法

`v-model="响应式数据"`

11.4 需求

使用双向绑定实现以下需求:

1. 点击登录按钮获取表单中的内容
2. 点击重置按钮清空表单中的内容

```
const loginForm = reactive({
  username: '',
  password: ''
})
```



用户:

密码:

11.5 完整代码

```
1  <script setup>
2    // 导入 reactive 响应式函数
3    import { reactive } from 'vue'
4
5    // 登录表单对象
6    const loginForm = reactive({
7      username: '',
8      password: ''
9    })
10
11 </script>
12 <template>
13   <div class="login-box">
14     账户: <input type="text" v-model="loginForm.username"/><br /><br />
15     密码: <input type="password" v-model="loginForm.password"/><br /><br />
16     <button>登 录</button>
17     <button>重 置</button>
18   </div>
19 </template>
```

12. 案例代办任务列表



项目克隆地址:

[GitHub - tastejs/todomvc-app-template](https://github.com/tastejs/todomvc-app-template): Template used for creating TodoMVC apps

12.1 静态代码(可复制)

新建 styles/index.css

代码块

```
1  @charset 'utf-8';
2
3  html,
4  body {
5      margin: 0;
6      padding: 0;
7  }
8
9  hr {
10     margin: 20px 0;
11     border: 0;
12     border-top: 1px dashed #c5c5c5;
13     border-bottom: 1px dashed #f7f7f7;
14 }
15
16 .learn a {
17     font-weight: normal;
18     text-decoration: none;
19     color: #b83f45;
20 }
21
22 .learn a:hover {
23     text-decoration: underline;
24     color: #787e7e;
25 }
26
27 .learn h3,
28 .learn h4,
29 .learn h5 {
30     margin: 10px 0;
31     font-weight: 500;
32     line-height: 1.2;
33     color: #000;
34 }
35
36 .learn h3 {
37     font-size: 24px;
38 }
39
40 .learn h4 {
41     font-size: 18px;
42 }
43
```



```
44 .learn h5 {
45   margin-bottom: 0;
46   font-size: 14px;
47 }
48
49 .learn ul {
50   padding: 0;
51   margin: 0 0 30px 25px;
52 }
53
54 .learn li {
55   line-height: 20px;
56 }
57
58 .learn p {
59   font-size: 15px;
60   font-weight: 300;
61   line-height: 1.3;
62   margin-top: 0;
63   margin-bottom: 0;
64 }
65
66 #issue-count {
67   display: none;
68 }
69
70 .quote {
71   border: none;
72   margin: 20px 0 60px 0;
73 }
74
75 .quote p {
76   font-style: italic;
77 }
78
79 .quote p:before {
80   content: '“';
81   font-size: 50px;
82   opacity: 0.15;
83   position: absolute;
84   top: -20px;
85   left: 3px;
86 }
87
88 .quote p:after {
```

```

1  <script setup></script>
2
3  <template>
4    <section class="todoapp">
5      <header class="header">
6        <h1>todos</h1>
7        <input
8          class="new-todo"
9          placeholder="What needs to be done?"
10         autofocus />
11      </header>
12      <section class="main">
13        <ul class="todo-list">
14          <li>
15            <div class="view">
16              <label>烫个卷毛</label>
17              <button class="destroy"></button>
18            </div>
19          </li>
20        </ul>
21      </section>
22      <footer class="footer">
23        <span class="todo-count"><strong>0</strong> item all</span>
24        <button class="clear-completed">Clear all</button>
25      </footer>
26    </section>
27  </template>

```

12.2 功能需求

1. 列表渲染

```

1  <script setup>
2    import { ref } from 'vue'
3
4    // 代办任务列表
5    const todoList = ref([
6      { id: 321, name: '吃饭饭', finished: false },
7      { id: 666, name: '睡觉觉', finished: true },
8      { id: 195, name: '打豆豆', finished: false }
9    ])
10 </script>
11
12 <template>

```

```
13 <ul class="todo-list">
14   <li
15     v-for="(item, index) in todoList"
16     :key="item.id">
17     <div class="view">
18       <label>{{ item.name }}</label>
19     </div>
20   </li>
21 </ul>
22 </template>
```

2. 添加功能

```
1 <script setup>
2   // 任务名称
3   const title = ref('')
4
5   // 新增
6   const add = () => {
7     // 去除首尾空格
8     const name = title.value.trim()
9     // 非空判断
10    if (!name) {
11      alert('任务名称不能为空')
12      title.value = ''
13      return
14    }
15    // 给数组新增一个对象
16    todoList.value.push({
17      id: Date.now(), // 时间戳
18      name,
19      finished: false
20    })
21    // 清空输入框
22    title.value = ''
23  }
24 </script>
25
26 <template>
27   <input
28     v-model="title"
29     @keydown.enter="add" />
30 </template>
```

3. 删除功能

```
1  <script setup>
2    // 删除
3    const del = (i) => {
4      if (window.confirm('确认删除么?')) {
5        todoList.value.splice(i, 1)
6      }
7    }
8  </script>
9
10 <template>
11   <li v-for="(item, index) in todoList">
12     <button @click="del(index)"></button>
13   </li>
14 </template>
```

4. 底部统计和清空

```
1  <script setup>
2    // 清空
3    const clearAll = () => {
4      if (window.confirm('确认清空所有么?')) {
5        todoList.value = []
6      }
7    }
8  </script>
9
10 <template>
11   <footer class="footer" v-if="todoList.length > 0">
12     <span class="todo-count">
13       <strong>{{ todoList.length }}</strong> item all</span>
14     >
15     <button
16       class="clear-completed"
17       @click="clearAll">
18       Clear all
19     </button>
20   </footer>
21 </template>
```